

学号：22920202200773

姓名：孟媛

实验三 列主元消去法求解线性方程组

一、实验目的

使用列主元消去法求解线性方程组。

二、数值计算原理

利用初等行变换将系数矩阵的增广矩阵的对角线下方元素从第一列依次到倒数第三列变化成 0，就可以通过从最后一行到第一行回代求得方程组的解。在此基础上，在每次将对角线下某一列清零时，都将这一列的对角线上的元素所在的行和这一列上对角线及以下元素中有着最大绝对值的元素所在的行进行交换。

三、源程序介绍

函数接受两个参数：方程组 $Ax = b$ 的 A 矩阵和 b 矩阵。

计算过程：

(1) 得到增广矩阵[A, b]

(2) 选主元：选 $a(k:n, k)$ 上绝对值最大的数所在的行和第 k 行进行交换

(3) 消元：利用初等变换 $a_{rj} = a_{rj} - a_{rj} * a_{rr} / a_{kk} (j = k, k + 1 \dots n)$ 进行消元

$$\begin{cases} x_n = b_n / a_{nn} \\ x_i = (b_i - \sum_{j=i+1}^n a_{ij} x_j) / a_{ii} \quad (i = n-1, \dots, 2, 1) \end{cases}$$

(4) 回代：利用公式

回代得到 x

```

1  #高斯列主元消去法求线性方程组的解
2  import numpy
3  from scipy.optimize import fsolve
4
5  def fgauss(A, b):
6      n = A.shape[0]
7      m_U = numpy.hstack((A, b.T)) #增广矩阵
8      ra = numpy.linalg.matrix_rank(A) #系数矩阵的秩
9      rb = numpy.linalg.matrix_rank(m_U) #增广矩阵的秩
10     if rb - ra > 0:
11         print("系数矩阵与增广矩阵秩不同,因而无解")
12         return
13     if ra == rb:
14         if ra == n:
15             x = numpy.mat(numpy.zeros(m_U.shape[0], dtype=float))
16             for p in range(0, n):
17                 print('此次行变换前增广矩阵为:')
18                 print(m_U)
19                 j = (abs(m_U[p:n, p])).argmax()
20                 print(f'第{p + 1}行与第{j + p + 1}行交换')
21                 temp = m_U[p, :].copy()
22                 m_U[p, :] = m_U[j + p, :]
23                 m_U[j + p, :] = temp
24                 print('行变换后的增广矩阵为:')
25                 print(m_U)
26                 print()
27                 for k in range(p + 1, n):
28                     m = m_U[k, p] / m_U[p, p]
29                     m_U[k, p:n + 1] = m_U[k, p:n + 1] - m * m_U[p, p:n + 1]
30             b1 = m_U[0:n, n]
31             a1 = m_U[0:n, 0:n]
32             print('增广矩阵为')
33             print(m_U)
34             x[0, n-1] = b1[n-1]/a1[n-1, n-1]
35             for i in range(n - 2, -1, -1):
36                 try:
37                     x[0, i] = (b1[i] - numpy.sum(numpy.multiply(a1[i, i + 1:n], x[0, i + 1:n]))) / (a1[i, i])
38                 except:
39                     print("异常错误!")
40             print('自编函数求得线性方程组的解为: ')
41             print(x)
42
43     if __name__ == '__main__':
44         A = numpy.mat([[12, -3, 3],
45                        [-18, 3, -1],
46                        [1, 1, 1]],
47                        dtype=float)
48         b = numpy.mat([15, -15, 6])
49         fgauss(A, b)
50         A = numpy.array([12, -3, 3, -18, 3, -1, 1, 1, 1],
51                          dtype = float).reshape(3, 3)
52         b = numpy.array([15, -15, 6])
53         print("自带函数求得方程组的解为:\n", numpy.linalg.solve(A, b))
54

```

四、程序执行情况

4.1 书上给的例子:

```

A = numpy.array([12, -3, 3, -18, 3, -1, 1, 1, 1],
                 dtype = float).reshape(3, 3)
b = numpy.array([15, -15, 6])

```

此次行变换前增广矩阵为：

```
[[ 12.  -3.   3.  15.]  
 [-18.   3.  -1. -15.]  
 [  1.   1.   1.   6.]]
```

第1行与第2行交换

行变换后的增广矩阵为：

```
[[ -18.   3.  -1. -15.]  
 [  12.  -3.   3.  15.]  
 [  1.   1.   1.   6.]]
```

此次行变换前增广矩阵为：

```
[[ -18.         3.         -1.         -15.         ]  
 [  0.         -1.         2.33333333  5.         ]  
 [  0.         1.16666667  0.94444444  5.16666667]]
```

第2行与第3行交换

行变换后的增广矩阵为：

```
[[ -18.         3.         -1.         -15.         ]  
 [  0.         1.16666667  0.94444444  5.16666667]  
 [  0.         -1.         2.33333333  5.         ]]
```

此次行变换前增广矩阵为：

```
[[ -18.         3.         -1.         -15.         ]  
 [  0.         1.16666667  0.94444444  5.16666667]  
 [  0.         0.         3.14285714  9.42857143]]
```

第3行与第3行交换

行变换后的增广矩阵为：

```
[[ -18.         3.         -1.         -15.         ]  
 [  0.         1.16666667  0.94444444  5.16666667]  
 [  0.         0.         3.14285714  9.42857143]]
```

增广矩阵为

```
[[ -18.         3.         -1.         -15.         ]  
 [  0.         1.16666667  0.94444444  5.16666667]  
 [  0.         0.         3.14285714  9.42857143]]
```

自编函数求得线性方程组的解为：

```
[[1. 2. 3.]]
```

自带函数求得方程组的解为：

```
[1. 2. 3.]
```

PS D:\myy\Math_Analysis> □

4.2 书上未给的例子

```
A = numpy.array([2, 3, 10, 5, 1, 1, 6, 2, 5, 1, 3, 2, 1, 1, 3, 4],  
                dtype = float).reshape(4, 4)  
b = numpy.array([2, 1, -3, -3])
```

此次行变换前增广矩阵为:

```
[[ 2.  3. 10.  5.  2.]  
 [ 1.  1.  6.  2.  1.]  
 [ 5.  1.  3.  2. -3.]  
 [ 1.  1.  3.  4. -3.]]
```

第1行与第3行交换

行变换后的增广矩阵为:

```
[[ 5.  1.  3.  2. -3.]  
 [ 1.  1.  6.  2.  1.]  
 [ 2.  3. 10.  5.  2.]  
 [ 1.  1.  3.  4. -3.]]
```

此次行变换前增广矩阵为:

```
[[ 5.  1.  3.  2. -3. ]  
 [ 0.  0.8 5.4  1.6 1.6]  
 [ 0.  2.6 8.8  4.2 3.2]  
 [ 0.  0.8 2.4  3.6 -2.4]]
```

第2行与第3行交换

行变换后的增广矩阵为:

```
[[ 5.  1.  3.  2. -3. ]  
 [ 0.  2.6 8.8  4.2 3.2]  
 [ 0.  0.8 5.4  1.6 1.6]  
 [ 0.  0.8 2.4  3.6 -2.4]]
```

此次行变换前增广矩阵为:

```
[[ 5.  1.  3.  2.  -3.  ]  
 [ 0.  2.6 8.8  4.2  3.2  ]  
 [ 0.  0.  2.69230769 0.30769231 0.61538462]  
 [ 0.  0. -0.30769231 2.30769231 -3.38461538]]
```

第3行与第3行交换

行变换后的增广矩阵为:

```
[[ 5.  1.  3.  2.  -3.  ]  
 [ 0.  2.6 8.8  4.2  3.2  ]  
 [ 0.  0.  2.69230769 0.30769231 0.61538462]  
 [ 0.  0. -0.30769231 2.30769231 -3.38461538]]
```

此次行变换前增广矩阵为:

```
[[ 5.  1.  3.  2.  -3.  ]  
 [ 0.  2.6 8.8  4.2  3.2  ]  
 [ 0.  0.  2.69230769 0.30769231 0.61538462]  
 [ 0.  0.  0.  2.34285714 -3.31428571]]
```

第4行与第4行交换

行变换后的增广矩阵为:

```
[[ 5.  1.  3.  2.  -3.  ]  
 [ 0.  2.6 8.8  4.2  3.2  ]  
 [ 0.  0.  2.69230769 0.30769231 0.61538462]  
 [ 0.  0.  0.  2.34285714 -3.31428571]]
```

增广矩阵为

```
[[ 5.  1.  3.  2.  -3.  ]  
 [ 0.  2.6 8.8  4.2  3.2  ]  
 [ 0.  0.  2.69230769 0.30769231 0.61538462]  
 [ 0.  0.  0.  2.34285714 -3.31428571]]
```

自编函数求得线性方程组的解为:

```
[[ -0.70731707  2.19512195  0.3902439 -1.41463415]]
```

自带函数求得方程组的解为:

```
[-0.70731707  2.19512195  0.3902439 -1.41463415]
```

PS D:\myy\Math_Analysis> □

五、结果分析

自编函数与工具函数求得结果一致,说明列主元消去法是一种稳定的求解线性方程组的方法,有着较好的鲁棒性,对很多病态方程组仍然有效。但自编函数的过程较为复杂,极其容易出错。

六、实验体会

此次实验，我认识到了列主元消去法的优秀性能，同时学习完成了自编函数的代码。我对列主元消去法的理解更加深入，总体来说还是一种比较好的方法。